Métodos de Análisis de Datos 1

Clase 4: Calidad de Datos

Universidad Nacional del Sur

Contenidos de hoy

¿Qué es la calidad de datos?

Definición

Los datos de calidad son aquellos que son **aptos para su uso previsto** en operaciones, toma de decisiones y planificación.

¿Qué es la calidad de datos?

Definición

Los datos de calidad son aquellos que son **aptos para su uso previsto** en operaciones, toma de decisiones y planificación.

Dimensiones de la calidad

1. **Completitud:** ¿Hay valores faltantes?
2. **Consistencia:** ¿Los datos son coherentes entre sí?
3. **Precisión:** ¿Los datos son correctos?
4. **Actualidad:** ¿Los datos están actualizados?
5. **Validez:** ¿Los datos cumplen reglas de negocio?

La realidad de los datos

Dato importante

En la práctica, **la mayoría de los datos están sucios**

- 60-80% del tiempo de un proyecto se gasta en limpieza de datos
- Los datos crudos casi nunca están listos para análisis
- La calidad de los datos determina la calidad del análisis

La realidad de los datos

Dato importante

En la práctica, **la mayoría de los datos están sucios**

- 60-80% del tiempo de un proyecto se gasta en limpieza de datos
- Los datos crudos casi nunca están listos para análisis
- La calidad de los datos determina la calidad del análisis

Regla de oro

Garbage in, garbage out

Si los datos de entrada son malos, los resultados también lo serán

Valores faltantes: El problema más común

¿Qué son?

Valores que deberían estar presentes pero no lo están

Representaciones comunes

- Python: `NaN`, `None`
- R: `NA`, `NaN`, `NULL`
- CSV: celdas vacías, `NA`, `?`, `-999`

Valores faltantes: El problema más común

¿Qué son?

Valores que deberían estar presentes pero no lo están

Representaciones comunes

- Python: `NaN`, `None`
- R: `NA`, `NaN`, `NULL`
- CSV: celdas vacías, `NA`, `?`, `-999`

¿Por qué faltan?

- Error en la recolección
- Sensor/dispositivo falló
- Usuario no respondió
- No aplica (ej: salario de un desempleado)
- Datos perdidos en transferencia

Tipos de valores faltantes

MCAR: Missing Completely At Random

La ausencia **no tiene relación** con ninguna variable

Ejemplo: Un sensor falla aleatoriamente

Tipos de valores faltantes

MCAR: Missing Completely At Random

La ausencia **no tiene relación** con ninguna variable
Ejemplo: Un sensor falla aleatoriamente

MAR: Missing At Random

La ausencia está relacionada con **variables observadas**
Ejemplo: Personas jóvenes no responden pregunta sobre ingresos (relacionado con edad, no con ingresos)

Tipos de valores faltantes

MCAR: Missing Completely At Random

La ausencia **no tiene relación** con ninguna variable
Ejemplo: Un sensor falla aleatoriamente

MAR: Missing At Random

La ausencia está relacionada con **variables observadas**
Ejemplo: Personas jóvenes no responden pregunta sobre ingresos (relacionado con edad, no con ingresos)

MNAR: Missing Not At Random

La ausencia está relacionada con el **valor faltante mismo**
Ejemplo: Personas con ingresos altos no responden pregunta sobre ingresos

El tipo de missingness determina qué estrategia usar

Detección de valores faltantes

Python:

```python
# Contar NAs por columna
df.isnull().sum()

# Porcentaje
(df.isnull().sum() / len(df)) *
    100

# Visualizar patr n
import missingno as msno
msno.matrix(df)
msno.heatmap(df)
```

R:

```r
# Contar NAs
colSums(is.na(df))

# Porcentaje
colMeans(is.na(df)) * 100

# Visualizar
library(naniar)
vis_miss(df)
gg_miss_var(df)
```

Estrategias para valores faltantes

1. Eliminación

Borrar filas o columnas con valores faltantes

2. Imputación simple

Reemplazar con media, mediana, moda o constante

3. Imputación avanzada

Usar algoritmos para predecir valores faltantes (KNN, MICE, regresión)

¿Cuál elegir?

Depende de:

- Porcentaje de valores faltantes
- Tipo de missingness (MCAR, MAR, MNAR)

Estrategia 1: Eliminación

Ventajas:

- Simple
- Rápido
- No introduce sesgo si es MCAR

Desventajas:

- Pérdida de información
- Reduce tamaño del dataset
- Sesgos si no es MCAR

Regla general

Solo eliminar si:

- Menos del 5% de valores faltantes
- Los datos son MCAR
- El dataset es suficientemente grande

Eliminación: Código

Python:

```python
# Eliminar filas con cualquier NA
df_clean = df.dropna()

# Eliminar filas donde
# columnas espec ficas tienen NA
df_clean = df.dropna(
    subset=['col1', 'col2']
)

# Eliminar columnas
# con >50% de NAs
umbral = len(df) * 0.5
df_clean = df.dropna(
    thresh=umbral,
    axis=1
)
```

R:

```r
# Eliminar filas con NA
df_clean <- na.omit(df)

# Columnas espec ficas
df_clean <- df %>%
  filter(
    !is.na(col1),
    !is.na(col2)
  )

# Eliminar columnas
# con >50% NAs
umbral <- nrow(df) * 0.5
df_clean <- df %>%
  select(where(
    ~ sum(is.na(.)) < umbral
```

Estrategia 2: Imputación simple

Opciones

- **Media:** Para variables numéricas, distribución simétrica
- **Mediana:** Para variables numéricas con outliers
- **Moda:** Para variables categóricas
- **Constante:** 0, "desconocido", etc.

Estrategia 2: Imputación simple

Opciones

- **Media:** Para variables numéricas, distribución simétrica
- **Mediana:** Para variables numéricas con outliers
- **Moda:** Para variables categóricas
- **Constante:** 0, "desconocido", etc.

Ventajas:

- Simple y rápido
- No reduce tamaño
- Funciona razonablemente bien

Desventajas:

- Subestima varianza
- No considera relaciones
- Puede introducir sesgo

Imputación simple: Código

Python:

```python
from sklearn.impute import
    SimpleImputer

# Media
imputer = SimpleImputer(
    strategy='mean'
)
df[['col1', 'col2']] = imputer.
    fit_transform(
    df[['col1', 'col2']]
)

# Moda (m s frecuente)
imputer = SimpleImputer(
    strategy='most_frequent'
)
```

R:

```r
library(tidyverse)

# Media
df <- df %>%
  mutate(col1 = ifelse(
    is.na(col1),
    mean(col1, na.rm = TRUE),
    col1
  ))

# Mediana (todas num ricas)
df <- df %>%
  mutate(across(
    where(is.numeric),
    ~replace_na(., median(., na.rm
        = TRUE))
```

Estrategia 3: Imputación avanzada

Métodos

- **KNN:** Imputa usando valores de observaciones similares
- **Regresión:** Predice valores faltantes usando otras variables
- **MICE:** Multiple Imputation by Chained Equations
- **Interpolación:** Para series temporales

Estrategia 3: Imputación avanzada

Métodos

- **KNN:** Imputa usando valores de observaciones similares
- **Regresión:** Predice valores faltantes usando otras variables
- **MICE:** Multiple Imputation by Chained Equations
- **Interpolación:** Para series temporales

¿Cuándo usar?

- Cuando hay relaciones fuertes entre variables
- Dataset mediano/grande
- Valores faltantes MAR
- Análisis requiere alta precisión

KNN Imputation

Idea

Usar los K vecinos más cercanos para estimar el valor faltante

Python:

```python
from sklearn.impute import KNNImputer

# Imputar con 5 vecinos
imputer = KNNImputer(n_neighbors=5)
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

R:

```r
library(mice)

# MICE (más sofisticado que KNN simple)
```

Duplicados: Registros repetidos

¿Qué son duplicados?

Filas que aparecen más de una vez en el dataset

¿Por qué ocurren?

- Error en la recolección (doble entrada)
- Múltiples fuentes sin deduplicación
- Problemas en joins/merges
- Errores de sistema

Duplicados: Registros repetidos

¿Qué son duplicados?

Filas que aparecen más de una vez en el dataset

¿Por qué ocurren?

- Error en la recolección (doble entrada)
- Múltiples fuentes sin deduplicación
- Problemas en joins/merges
- Errores de sistema

Impacto

- Sesgan estadísticas (media, totales)
- Inflan el tamaño del dataset
- Problemas en modelos de ML
- Resultados incorrectos

Detección de duplicados

Python:

```python
# Todas las columnas
duplicados = df.duplicated()
print(f"Total: {duplicados.sum()}"
    )

# Columnas específicas
duplicados = df.duplicated(
    subset=['id', 'fecha']
)

# Ver duplicados
df[duplicados]
```

R:

```r
# Detectar duplicados
duplicados <- duplicated(df)
cat("Total:", sum(duplicados))

# Columnas específicas
duplicados <- df %>%
  duplicated(by = c('id', 'fecha')
    )

# Ver duplicados
df[duplicados, ]
```

Eliminación de duplicados

Python:

```python
# Eliminar (mantiene primero)
df_clean = df.drop_duplicates()

# Por columnas espec ficas
df_clean = df.drop_duplicates(
    subset=['id', 'fecha']
)

# Mantener  ltimo
df_clean = df.drop_duplicates(
    keep='last'
)

# Ver cu les se eliminan
df_clean = df.drop_duplicates(
    keep=False
```

R:

```r
# Eliminar duplicados
df_clean <- distinct(df)

# Columnas espec ficas
df_clean <- df %>%
  distinct(id, fecha,
           .keep_all = TRUE)
```

Inconsistencias: Datos que no tienen sentido

Tipos de inconsistencias

1. **Valores fuera de rango:** Edad = -5, temperatura = 500°C
2. **Formatos incorrectos:** Fecha "32/13/2024"
3. **Inconsistencias entre variables:** Fecha nacimiento > Fecha graduación
4. **Errores tipográficos:** "Bahia Blanca", "B. Blanca", "Bahía blanca"
5. **Valores imposibles:** Porcentaje > 100%, probabilidad < 0

Detección de inconsistencias

Python:

```python
# Valores fuera de rango
print("Edades negativas:",
      (df['edad'] < 0).sum())
print("Edades >120:",
      (df['edad'] > 120).sum())

# Rangos
print(f"Min: {df['edad'].min()}")
print(f"Max: {df['edad'].max()}")

# Valores  nicos
print(df['ciudad'].value_counts())

# Entre columnas
inconsist = df[
    df['fecha_nac'] > df['
```

R:

```r
# Fuera de rango
cat("Negativas:",
    sum(df$edad < 0, na.rm = TRUE)
      )

# Rangos
cat("Min:", min(df$edad, na.rm =
   TRUE))
cat("Max:", max(df$edad, na.rm =
   TRUE))

# Valores  nicos
table(df$ciudad)

# Entre columnas
inconsist <- df %>%
```

Corrección: Estandarización de texto

Python:

```python
# Lowercase y trim
df['ciudad'] = df['ciudad'].str.
    lower().str.strip()

# Reemplazar variaciones
df['ciudad'] = df['ciudad'].
    replace({
     'b. blanca': 'bahia blanca',
     'bblanca': 'bahia blanca'
})

# Regex para reemplazar
df['ciudad'] = df['ciudad'].str.
    replace(
     r'bahia\s+blanca',
     'bahia blanca',
```

R:

```r
# Lowercase y trim
df <- df %>%
  mutate(ciudad = str_to_lower(
    str_trim(ciudad)
  ))

# Reemplazar
df <- df %>%
  mutate(ciudad = recode(ciudad,
    'b. blanca' = 'bahia blanca',
    'bblanca' = 'bahia blanca'
  ))
```

Corrección: Valores fuera de rango

Python:

```python
# Reemplazar con NA
df.loc[df['edad'] < 0, 'edad'] =
    np.nan
df.loc[df['edad'] > 120, 'edad'] =
     np.nan

# Clip (limitar valores)
df['edad'] = df['edad'].clip(
     lower=0,
     upper=120
)

# Filtrar filas v lidas
df_clean = df[
     (df['edad'] >= 0) &
     (df['edad'] <= 120)
```

R:

```r
# Reemplazar con NA
df <- df %>%
  mutate(edad = ifelse(
    edad < 0 | edad > 120,
    NA,
    edad
  ))

# Con pmin/pmax
df <- df %>%
  mutate(edad = pmin(
    pmax(edad, 0),
    120
  ))
```

Pipeline de limpieza de datos

1. **Cargar datos**
2. **Exploración inicial:** dimensiones, tipos, primeras filas
3. **Detectar valores faltantes:** porcentaje, patrón
4. **Decidir estrategia:** eliminar o imputar
5. **Detectar duplicados:** por todas las columnas o subset
6. **Eliminar duplicados**
7. **Detectar inconsistencias:** rangos, formatos, relaciones
8. **Corregir inconsistencias:** estandarizar, clip, reemplazar
9. **Verificación final:** re-explorar datos limpios
10. **Documentar:** qué se hizo y por qué

Resumen de la clase

- La **calidad de datos** tiene múltiples dimensiones
- Los **valores faltantes** son el problema más común
- Tipos de missingness: **MCAR, MAR, MNAR**
- Estrategias: **eliminación** (simple) o **imputación** (simple/avanzada)
- Los **duplicados** sesgan resultados y deben eliminarse
- Las **inconsistencias** requieren análisis cuidadoso
- La **estandarización** de texto es crucial
- **Documentar** todas las decisiones de limpieza

Próxima clase

Transformación de datos: tipos, variables derivadas, codificación de categóricas

Tarea

1. Tomar el dataset de la clase anterior
2. Realizar análisis de calidad:
   - ¿Hay valores faltantes? ¿Cuántos? ¿En qué columnas?
   - ¿Hay duplicados?
   - ¿Hay valores fuera de rango?
   - ¿Hay inconsistencias en texto?
3. Aplicar estrategias de limpieza apropiadas
4. Documentar en el notebook:
   - Qué problemas encontraste
   - Qué estrategias usaste
   - Por qué elegiste esas estrategias
   - Comparar antes vs después
5. Subir a GitHub

¿Preguntas?